

Spelsajt

Från auktoritativ spelmotor till levande 3D-presentation och trygg AI.

Målet är en modern play-money-sajt för blackjack och europeisk roulette. Servern bestämmer alltid regler, kort, roulettepocket och saldo. Frontend och AI får göra upplevelsen levande, men får aldrig ändra spelets lagar.



En mening att styra projektet efter

**Motorn skapar sanningen. Eventen bär sanningen.
Scenen gestaltar sanningen.**

Status idag: spelkärnor, fairness, v2-kontrakt och presentationsprojektor finns. Nästa stora steg är att koppla dem genom ett riktigt command-API, Supabase-transaktioner, auth och spelbara bord.

En tydlig maktordning

Varje lager har en enda uppgift. Det gör att Jakob kan bygga backend medan Emil bygger känsla, 3D och rörelse utan att båda behöver ändra samma logik.

1. Commands	Frontend skickar spelarens avsikt, exempelvis HIT, STAND eller ROULETTE_SPIN. Den skickar aldrig ett önskat resultat.
2. Spelmotor	En ren TypeScript-state-machine validerar fas och regler, tar injicerad fairness och returnerar nytt state, semantiska events och ledger-intents.
3. Transaktion	Servern sparar command, state, saldoändring, fairnessmetadata och eventsekvens atomiskt i Supabase. Ett commandId får bara verkställas en gång.
4. Eventström	Frontend får versionsstyrda GameEventV2 med stigande sekvensnummer. Vid avbrott hämtas snapshot och uppspelningen fortsätter.
5. Presentation	Animation Director mappar event till visuella intents. GLB-klipp, ljud och text kan bytas utan att spelregler eller utfall förändras.

Hård gräns

- Ingen produktionskod använder Math.random för spelutfall.
- Inga dolda blackjackkort skickas med rank, suit eller cardId.
- Dummygrafik på startsidan märks som dekoration och går aldrig genom spelstoren.

Regler före grafik

Blackjack och roulette ska vara två separata state-machines med samma yttre form. Båda tar ett state och ett validerat command och returnerar nytt state, events och bokföringsavsikt.

BLACKJACK	EUROPEISK ROULETTE
Faser: prepared > player turn > dealer turn > settled. Motorn äger sko, händer, tillåtna actions, split/double, dealerregler och payout. Fairness levererar en reproducerbar skoordning.	Faser: created > betting open > locked > spinning > settled. Motorn validerar alla bettyper, tar en injicerad pocket 0-36 och använder rulesetets payoutmatris. Frontend får aldrig välja landningsficka.

Fairness och saldo

- Server seed committas före rundan. Client seed och nonce ger reproducerbara bytes.
- Blackjack använder kryptografisk shuffle. Roulette mappar unbiased till exakt en pocket.
- PLAY transporteras som heltal. Motorn räknar inte kontosaldo; den returnerar ledger-intents.
- Supabase applicerar state och ledger i samma databastransaktion under RLS.

Regressionslås

Frysta rulesets och golden vectors gör att samma input alltid ger samma kortordning och pocket i Node och Web Crypto. Förändras ett observerbart utfall ska versionen ändras medvetet.

Eventstyrd scen, inte en andra motor

PresentationStore läser validerade event och bygger en visuell projektion. Animation Director väljer sedan ett godkänt visuellt intent. Emil kan byta modeller, kamera, easing och blendning utan att röra spelreglerna.

Exempel: blackjack

blackjack.card.dealt	Flyg ett kort från skon till angiven hand. Visa bara framsida när eventet innehåller ett publikt kort.
blackjack.card.revealed	Vänd exakt det tidigare dolda kortet. Lägg inte till eller hitta på ett nytt kort.
blackjack.turn.changed	Markera aktiv hand och aktivera bara de actions servern tillåter.
blackjack.hand.settled	Animera vinst, förlust eller push utifrån eventets outcome och payout.

Exempel: roulette

roulette.spin.started	Starta en neutral loop. Ingen landningsficka är vald av scenen.
roulette.result	Bromsa hjul och kula till pocket som servern skickade.
roulette.bet.settled	Markera varje bet och flytta marker visuellt utan att räkna om payout.

Responsiv hög standard

- Ett visuellt intent kan ha desktop-, mobil- och reduced-motion-variant.
- GLB-avatarer använder namngivna klipp och cross-fade; motorn känner aldrig till klippsnamn.
- Eventsekvensen kan spelas upp i textläge, enkel 2D eller full 3D med samma data.

AI ger personlighet, inte makt

AI ska ligga efter den deterministiska Reaction Planner. Systemet väljer först en säker cue från spelets event. AI får därefter formulera ton, kort replik eller variation inom en vitlistad ram.

Indata till AI	En liten säker kontext: exempelvis 'spelaren förlorade tredje handen i rad', vald persona, språk och tillåten ton. Inga hemliga seeds eller dolda kort.
Strukturerat svar	JSON enligt schema: cued, text, emotion, intensity och valfri gestureld från en godkänd lista. Fritext får aldrig innehålla spelbeslut.
Runtime	Spelögonblicket väntar inte på modellen. En deterministisk standardanimation startar direkt; ett godkänt AI-svar kan förfina nästa beat eller talrad.
Avataren	GLB-filen innehåller rigg och godkända klipp. AI redigerar inte modellen live, utan väljer parametrar till en blend tree: blick, kroppshållning, gesture och mimik.

Bra användning

- Croupiern kommenterar kort och naturligt efter ett tydligt spelutfall.
- Stämning, tempo och blick varierar utan att bordets tidslinje blir oförutsägbar.
- Offline kan AI hjälpa Emil skapa fler klipp, retargetingförslag och testscenarier.

Förbjudet

AI väljer aldrig kort, pocket, payout, saldo, tillåtna actions, command eller vinnare.

Bygg vertikalt, en riktig runda i taget

Nästa mål är inte fler animationer på startsidan. Det är en komplett kedja där en inloggad testspelare kan spela en riktig, serverstyrd runda och återansluta utan att state eller saldo går sönder.

A. Command-tjänst	Implementera POST /v2/tables/{tableId}/commands och snapshot-route. Validera Zod, auth, expectedRevision och commandId. Returnera accepted, replayed eller rejected.
B. Atomisk lagring	Koppla motortransition, state, eventsekvens, fairnessmetadata och PLAY-ledger i en Supabase-transaktion. Behåll RLS och negativa åtkomsttester.
C. Spelbart blackjack	Bygg enkla kontroller och en text/2D-vy först. Koppla sedan deal, reveal, split, dealerturn och settlement till Emils scene-intents.
D. Spelbar roulette	Bygg bettingmatta, läs bets, spin och settlement. Låt roulette.result styra exakt landning i 3D-hjulet.
E. Realtime och återanslutning	Streama events med stigande sequence. Vid gap eller reconnect hämtas snapshot och scenen återställs utan att spela om saldoeffekter.
F. AI-personlighet	Lägg till sist en strukturerad cue-tjänst med cache, timeout, fallback och kostnadsgräns. Den är presentation, aldrig en blockerande del av rundan.

Ett repo som håller ihop

All utveckling utgår från samma kontrakt och maskinläsbara regler. Dokumentation förklarar, men scheman, rulesets, fixtures och tester avgör om systemet faktiskt håller.

Varje vertikal slice är klar när

- commanden valideras och är idempotent; samma command kan inte debitera två gånger.
- motorn har transitionstester, edge cases och golden vectors där randomness påverkar.
- event, ack och snapshot valideras mot både Zod och genererat JSON Schema.
- ledger och state committas atomiskt samt skyddas av RLS.
- presentationsmappningen är uttömmande: varje event har cue eller explicit ignore.
- reduced motion, mobil vy, reconnect och dubbla commands provas.
- pnpm check och en riktig browserrunda är gröna före merge.

Praktisk ansvarsfördelning

Jakob / backend	Emil / frontend och 3D
Commands, auth, Supabase, motoradapter, fairness, ledger, eventordning och snapshots.	Spelkontroller, projection, scen, GLB-rigg, klipp, ljud, responsivitet och reduced motion.
Gemensamt	V2-kontrakt, inspelade scenarier, acceptanstester och systemcanvas. Kontraktsgap löses i små samordnade PR:er.

Slutmålet

En spelare kan förstå vad som händer, motorn kan bevisa varför det hände och Emil kan göra det vackert utan att presentationen någonsin blir en alternativ sanning.